Gokula Kanaa. J

240701149

CSE

Exp:1 Basic Network Commands

- Aim:

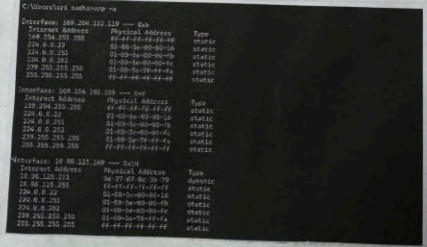
Study of various network commands used in linux and Windows.

Commands:

i) arp -a:-

ARP is short form of address resolution protocol which show the IP address of your command along with IP address and MAC address of your router.

Output:



2.

hostname

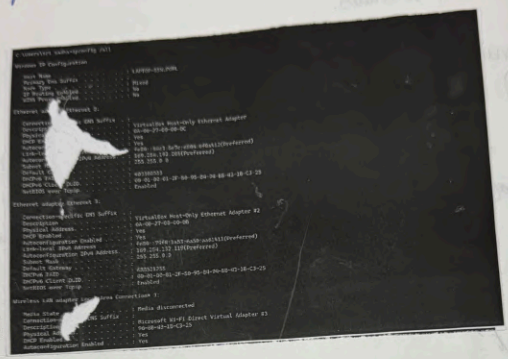
It displays the name of your Computer.

Sample outputs:

SRIRAM

3. IP Config /all :

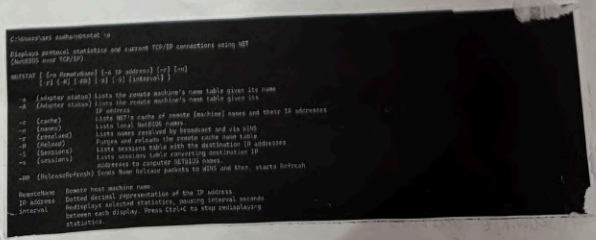
This Command displays detailed configured information about your TCP/IP Connection.



4. nbstart -a :

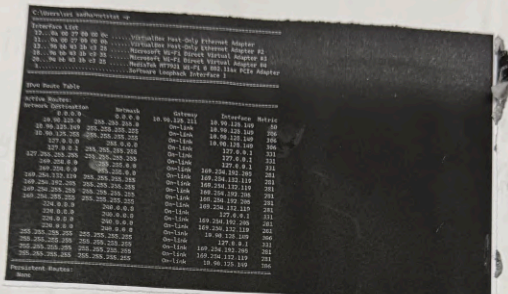
This Command helps solve problems with NBIOS name resolution.

Output:



5. netstat:

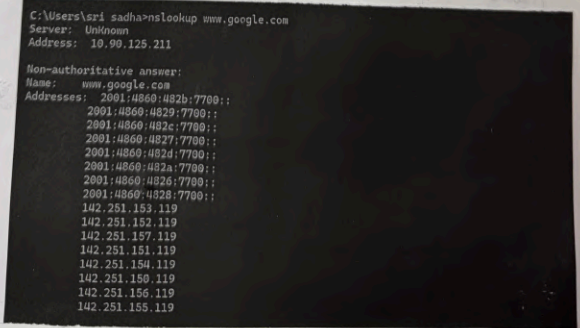
netstat displays a variety of statistics about a computer active TCP/IP connection. Its command line tool for monitoring network connections both incoming and outgoing as well as viewing routing table.



6. nslookup

It is used perform DNS lookups in linux.

Ex: nslookup www.google.com



7. Pathping

Pathping is unique to windows and is basically a combination of the ping and tracert commands.

```
C:\Users\sri sadha>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
   -g host-list        Loose source route along host-list.
   -h maximum_hops     Maximum number of hops to search for target.
   -i address          Use the specified source address.
   -n                  Do not resolve addresses to hostnames.
   -p period           Wait period milliseconds between pings.
   -q num_queries      Number of queries per hop.
   -w timeout          Wait timeout milliseconds for each reply.
   -4                  Force using IPv4.
   -6                  Force using IPv6.
```

8. Ping:

Commands in the cmd way to test

```
C:\Users\sri sadha>route print
===========================================================================
Interface List
  32...0a 00 27 00 00 0e ......VirtualBox Host-Only Ethernet Adapter
  17...0a 00 27 00 00 0b ......VirtualBox Host-Only Ethernet Adapter #2
  15...9e 9e 42 1b 42 25 ......Microsoft Wi-Fi Direct Virtual Adapter #2
  20...9e 9e 42 1b 42 24 ......Microsoft Wi-Fi Direct Virtual Adapter #4
  20...00 00 42 1b 42 20 ......Realtek RTL8723BE PCIe Adapter
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 Route Table

Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0      10.90.128.241   10.90.128.149     50
       10.90.128.0    255.255.255.0         On-link      10.90.128.149    306
      10.90.128.149  255.255.255.255         On-link      10.90.128.149    306
      10.90.128.255  255.255.255.255         On-link      10.90.128.149    306
        127.0.0.0        255.0.0.0         On-link         127.0.0.1    331
        127.0.0.1  255.255.255.255         On-link         127.0.0.1    331
  127.255.255.255  255.255.255.255         On-link         127.0.0.1    331
      169.254.0.0      255.255.0.0         On-link      169.254.192.205    281
      169.254.0.0      255.255.0.0         On-link      169.254.132.119    281
    169.254.132.119  255.255.255.255         On-link      169.254.132.119    281
    169.254.192.205  255.255.255.255         On-link      169.254.192.205    281
        224.0.0.0        240.0.0.0         On-link         127.0.0.1    331
        224.0.0.0        240.0.0.0         On-link      169.254.192.205    281
        224.0.0.0        240.0.0.0         On-link      169.254.132.119    281
  255.255.255.255  255.255.255.255         On-link         127.0.0.1    331
  255.255.255.255  255.255.255.255         On-link      169.254.192.205    281
  255.255.255.255  255.255.255.255         On-link      169.254.132.119    281
  255.255.255.255  255.255.255.255         On-link      10.90.128.149    306
===========================================================================
Persistent Routes:
  None

IPv6 Route Table
===========================================================================
Active Routes:
 If Metric Network Destination      Gateway
 32    306 ff00::/8                fe80::5c27:4744:fe8c:9b79
                                   On-link
 20     58 2409:4041:4d4a:a8cc::/62 On-link
 20    318 2409:4041:4d4a:a8cc:7623:1b21:7b77:c04e/128
                                   On-link
 20    306 2409:4041:4d4a:a8cc:7623:1b21:7b77:c04e/128
                                   On-link
 32    281 fe80::/64               On-link
 20    281 fe80::/64               On-link
```

9. Route

Route command is used to show / manipulate the routing table.

```
C:\Users\sri sadha>ping LAPTOP-55NLPGRL

Pinging LAPTOP-55NLPGRL [fe80::38f3:7a6c:ae35:d90d%20] with 32 bytes of data:
Reply from fe80::38f3:7a6c:ae35:d90d%20: time<1ms
Reply from fe80::38f3:7a6c:ae35:d90d%20: time<1ms
Reply from fe80::38f3:7a6c:ae35:d90d%20: time<1ms
Reply from fe80::38f3:7a6c:ae35:d90d%20: time<1ms

Ping statistics for fe80::38f3:7a6c:ae35:d90d%20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Import

1. IP:

IP Com
administra
up
Eg: ip

2. ip Con
The
sysadm
networ
Comma

3. netst
Ser
Outpu

4. tcp

Important Linux Networking Commands:

1. IP:

IP Command is one of the basic command every administrator will need in daily work from setting up new systems.

Eg: ip address show

2. ip Config

The ip Config command was a staple in many sysadmins tool belt for configuration and troubleshoot networking. It has since been replaced by the ip Commands discussed here

3. mtr:

Serves as a network diagnostic tool.

Output: mtr google.com

4. tcp dump

The tcp command designed for capturing and displaying packets.

Output: tcpdump -D

; dnf install -> tcpdump

3. Ping:

Tool to verify IP level connectivity to another TCP/IP Computer by sending internet Control message Protocol echo-request messages.

Output: Ping localhost.

Exp: 2

Aim:

To

Under

Cable typ

i) UTP

ii) STP

iii) SS

iv) C

Cab

v)

Exp: 2 Type of Network Cable

Aim:

To study of different types of network cable
Understand different types of network cable

Cable type	Category	Max data trans	Adv & Disadv	Application
i) UTP	3	10 bPs	i) cheaper ii) More prone to EMI	10-base T-ethernet
ii) STP	6, 6a	10 GbPs	i, shielded ii) Expensive	used in data centres
iii) SSTP	7	10 GbPs	i) Greater effort install	10G ethernet
iv) Coaxial Cable	RG-6 RG-59	10-100 Mbps	i, High bandwidth ii, Size in bulky	TV network
v) Fibre optics Cable	Single mode	100 kbps	i, High speed ii) Expensive	Max dist is 100m

b) Make your own ethernet Cross over cable.

Step 1: To start Construction of device, begin by the readings shield onto cable

step 2: Next, strip approximately 1. 5cm of cable shielding from both end. The crimping tool has around area.

Step 3: After you need to untangle the wire there should be four twisted pairs.

Step 4: Once the order is correct, bunch them together in line, and if there are any that stick out than snip them back to Create an even level.

Step 5: Next push the cable tight in the node at end of plug needs. to Jump over the cable shielding.

Step 6: After the wire are securely sitting inside the plug, insert it into the Crimping tool and push low.

Step 7: Lastly, Repeat for other end using diagram B.

Result:

The different type of network cable is studied.

Experiment-3

CISCO PACKET TRACER

Aim: To study the Packet trace tool installed and user interface.

a) UI overview:

The layout of packet trace is divided into served Components.

The Components are

i. Menu bar
ii. Main toolbox;
iii. Logical table
iv. Work Space
v. Real time table
vi. Network Component box
vii. Common tool box
viii. User - created.

Analyze behaviour of network device using CISCO PACKET TRACER simulation.

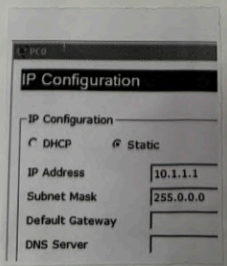
From network box drag & drop the Components

2. Click on Connection
   a. Click on Coffee straight through cable
   b. Connect PC0 to switch



3. Click on PC, connected to hub, go to desktop tab, click on IP Configuration and enter IP address and subnet mask. Here the default gateway and DNS server information is not needed as there are only two end devices connected in network.

4) Observe the flow of PDU from Source PC to destination PC by selecting the realtime mode.



5, Repeat the steps 3 to 5.

6, Observe how hub and switch are forwarding the PDU and write observation.

Result:

The CISCO PACKET TRACKER is successfully studied.

Experiment-4

LAN

Setup and Configure a LAN Closed Area Network using a switch and ethernet cables in your lab.

How to setup a LAN

Step 1: Plan and design an appropriate network topology, taking into account network requirement and equipment location.

Step 2: You can take 4 Computer, a switch with 8, 16 or 24 ports which is sufficient for network of these sizes and 4 ethernet cables.

Step 3: Connect your Computer to network switch via an ethernet cable, which is as simple as plugging one end of the ethernet cable into your Computer and the other end into your network switch.

Step 4: Assign IP address to your PC's

Step 5: Configure a network switch:

1. Connect your Computer to the switch:
- To access the switch's web interface, you

need to conne
using a ethe
2. Log in to
browser and
in the ad
login page
3. Configure
will be a
4. Assign
255.0.
Step
and othe
Comman
Step 8
showin
Step 8
Confi

need to connect your computer to the switch: To using a ethernet cable.

2. Log in to the web interface: Open a web browser and enter the IP address of the switch in the address bar. This should bring up the login page for the switch's web interface.

3. Configure basic settings: once you logged in you will be able to configure basic setting for switch.

4. Assign IP address as 10.1.1.5, subnet mask 255.0.0.0

Step 6: Check the connectivity between switch and other machines by using ping command in the command prompt of the device.

Step 7: Select a folder → go to properties → click sharing tab → share it with everyone on the same LAN.

Step 8: Try to access the shared folder from other computers of network.

Result:

The LAN is configured and setup successfully.

Exp: 5 Experiment on Packet Capture tool wireshark

Aim:

Experiments on packet capture tool: wireshark packet sniffer

-> Sniffer messages being sent/received from/by your computer.

-> Store and display the contents of various protocol field in the message.

Procedure:

-> Select local area connections in wireshark

-> Go to Capture option

-> Then select stop Capture automatically after 100 packets.

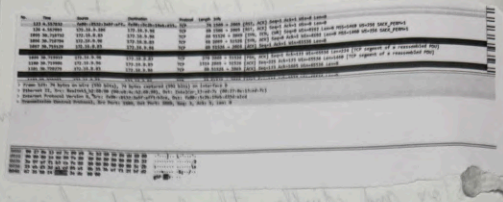
-> Then click start capture

-> Save the packets

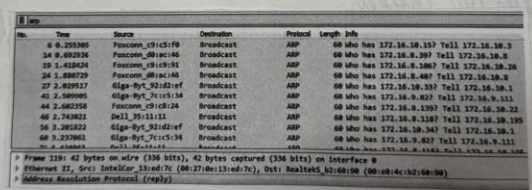
Output:

1. Create a filter only to display TCP packets, inspect the packet

2. Create a filter only to display ARP packets.

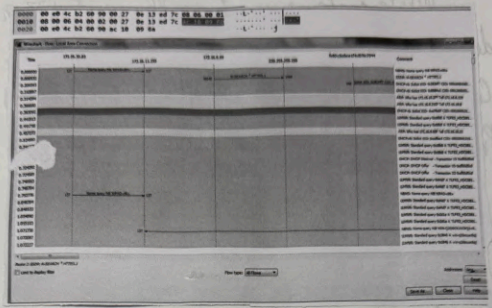


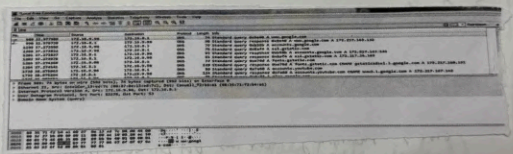
3. Create a filter only to display DNS Packets





4. Create a filter to display only DHCP packets





Result:

The experiment on wireshark is completed successfully.

Experiment: 6 Hamming

Aim:

Write a program to implement error detection and Correction using hamming Code Concept.

Procedure:

→ Create a Sender program with below features.

1, Input to Center file should be a text.

2, Apply hamming Code Concept on binary data

3) Save this output

→ Create a receiver program with below features.

1. Receiver program should need the input

2. Apply hamming Code as on binary data.

3. If there is error, display position of error.

```
def calculate_parity_bits(data_bits):
    r=0
    while (2**r) < (data_bits + r + 1):
        r += 1
    return r
def hamming_encode(data):
    m = len(data)
    r = calculate_parity_bits(m)
    total_bits = m+r
    hamming = [0]*(total_bits+1)
    data_index = 0
for position in range(1, total_bits+1):
    if position & (position-1)
    hamming[position] = int(data[data_index])
    data_index += 1
    parity_position = 1
    while parity_position <= total_bits
        parity = 0

For position in range(1, total_bits+1):
    if position & parity_positions:
        parity ^= hamming[position]
```

```
    text = file.read()
print("Original text")
print(text)
with open("channel.txt", "w") as channel:
for character in text
    channel.write(encode + " ")
print("\n Hamming encoded data saved to
                    channel.txt")
```

Receiver Code:

```
    def hamming_decode(block)
    total_bits = len(block)
    hamming = [0]
for bit in block
    hamming.append(int(bit))]
    error_position = 0
    family_position = 1
    while parity_position <= total_bits: parity = 0
for position in range(1, total_bits+1):
    if position & parity_position:
    parity ^= hamming[position])
```

```
        data += str[hamming[position]]
    return character
with open("channel.txt", "r") as channel
print("Received data:")
result = ""
for block in data:
    character = hamming.decode(block)
    result += character
    print("\n Original message after
            Correction:")
    print(result)
```

Result:

The Program has been executed successfully.

Flow Control of Data link Layer:

Experiment 7

Aim:

Write a program to implement flow control at data link layer using sliding window Protocol.

Code Program:

```
#include <stdio.h>
#include <stdlib>
#include <string.h>
#include <width>
#define MAX 100
struct frame {
    int frame_no;
    char data;
}
int main() {
  struct frame frame(MAX);
  int windowsize, n, i, start=0;
  char message[MAX];
  File fp;
  int ack;
  scanf("%d", &windowsize);
  scanf("%s", &message);
  n = strlen(message);
```

```
while [start<n) {
 int end = start + windowsize;
 if (end > n)
    end = n;
 fp = fopen ("Sender - Buffer", "w");
 if (fp = = NULL) {
     Print ("error");
     return;
  }
  for(i = start; i < end; i++) {
   { Printf(fp, "%d %c\n, frames[i], frame-no,
                          frames[i] . data);
  Print ("Frame No: %d DATA: %c\n",
          frame[i] frame no, frame [i] . data );
 }
 False [fp];
 Printf ("Frames stored in sender buffer");
 fp = fopen ("Receiver Buffer, "r");
     if (fp = = NULL) {
        Continue;
 }
 fscanf (fp = "%d", &ack);
 fclose (fp);
 Printf( "Received ACK/NACK %d\n, "ack);
 return 0;
}
```

Receiver Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    FILE *fp, *out;
    int frameNo
    char data;
    int expected = 1;
    while (1) {
      fp = fopen ("Sender Buffer", "r");
      if (fp == NULL) {
      printf ("Waiting");
      continue;
  }
while (fp, "%d %c", &frameNo, &data) {
if (frameNo == expected) {
out = fopen ("Receiver Buffer", "w");
if (out == NULL) {
      fclose (Fp);
      return 0;
    }
}
```

```
    fclose(fr);
    return 0;
  }
  return 0;
}
```

Result:

The Code has been executed successfully.

Experiment - 8

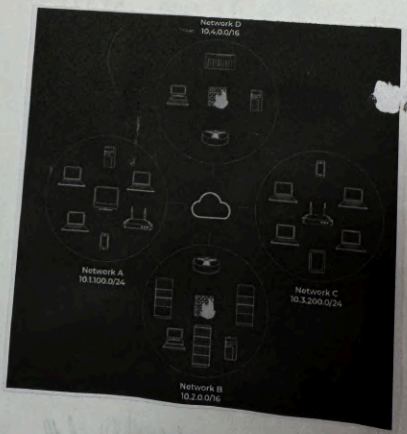
Aim: To discover live host using Nmap Scans (ARP, ICMP, TCP/UDP) on tryhackme Platform room Nmap Live Host.

Introduction:

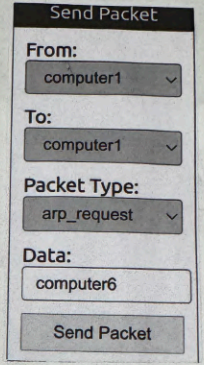
When targeting a networks we need an efficient tool to handle repetitive tasks.

This room is First of four in Nmap Lives.

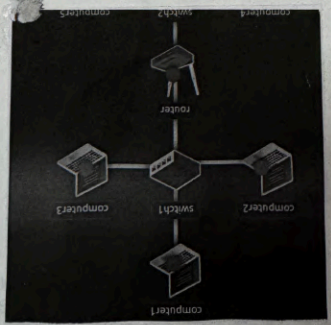
* Nmap live host Discovery.
* Nmap Advanced Port Scans.



The figure displays 2 types of subnets:
+ /16 subnets
+ /24 subnets



from Computer to Computer,
Packet type: "ARP required"



Send Packet

From:
computer4

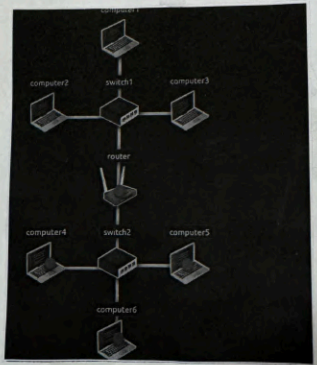
To:
computer4

Packet Type:
arp_request

Data:
computer6

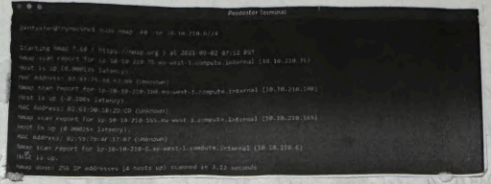
Send Packet

* from Computer 4 to Computer 4
* Packet type = "ARP Required"



Nmap Host discovery using ARP:

When a Privileged user tried to scan targets on local networks, Nmap uses ARP Required.



Nmap host discovery using ICMP.

A Simple method to identify live host on target network is by bringing each IP address and checking for responses.

UDP ping:

While Sending UDP packets to open UDP Ports may not trigger a response, sending them to close UDP Ports.

Result:

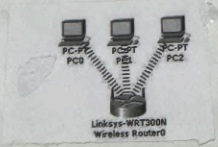
The result has been Completed successfully.

Experiment - 8(b)

Aim:

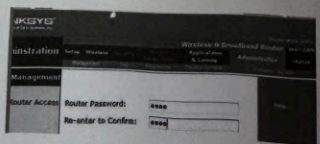
Configuration of wireless LAN using CISCO packet diagram.

Design a topology with three PC connected to Linksys window routers

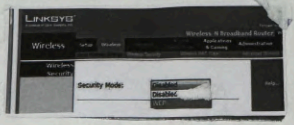


Procedure:

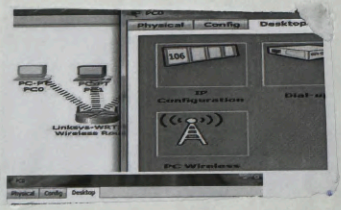
Step 1: Click on wireless router, Select admin tab from the menu.



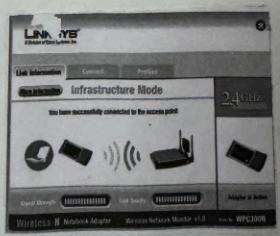
Step 2:
Now select wireless security & change security mode to WEP



Step 3: set key, to 0 1 2 3 4 5 6 7 8 9.



Step 4: Connect PCs From wireless Router.



Step 5: Click on connect tab & click on refresh button

Result:

The Configuration has been completed successfully

Experiment - 9

Aim: Implementation of subnetting in CISCO packet tracer simulator

Classless IP subconnecting is a technique that allows for more efficient use of IP address by allowing for subnet masks.

Creating a network topology

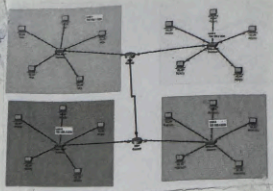
To create a network topology in packet trace, select "New" button in top, then select Network & generic.

Adding the devices:

To add a device, select the device from bottom left and drag it onto networks topology.

Subnetting:

To subnet the networks address of 192.168.1.0/24 to provide enough space for atleast 5 address for an end devices, switch and router.



Configuring the devices:

Right Click on router and select "CLI" This will open by Configurating the Router - Right click. CLI enter following commands

enable
interface FastEthernet 0/0
ip address [IP address] [subnet mask]
no shutdown
exit

interface Fast Ethernet 0/.

ip address [IP address] [subnet mask]
no shutdown.
exit.

Result:
The implementation has been completed successfully.

Experiment - 10(a)

Aim: Internetworking with Routers in CISCO PACKET TRACER simulation.

Procedure:

Step 1: Configure router

Step 2: Configure PC's

Step 3: Connecting PC with router

Router Configuration table

Device Name	IP address ethernet 0/0	Subnet mask	IP address ethernet 0/1	Subnet mask
Router	192.163.10.1	255.255.255.0	192.168.20.1	255.255.255.0

PC Configuration table:

Device Name	IP address	Subnet mask	Gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

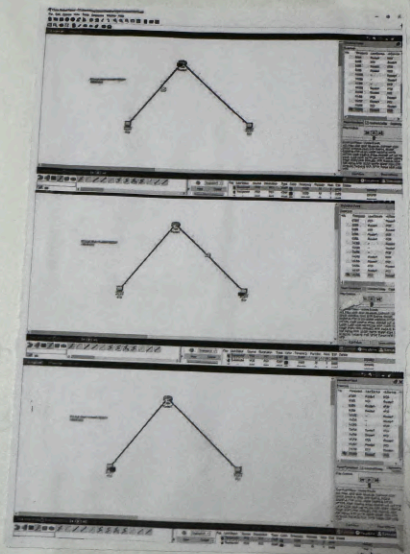
in CISCO

	Subnet Mask
.1	255.255.255.0

teway

168.10.1

68.20.1

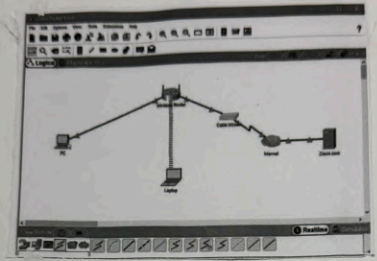


Result:

The internetworking with router has been completed successfully.

Experiment-10(b)

Aim: Design and Configure an internetworking using wireless routes, DHCP servers and internet cloud.

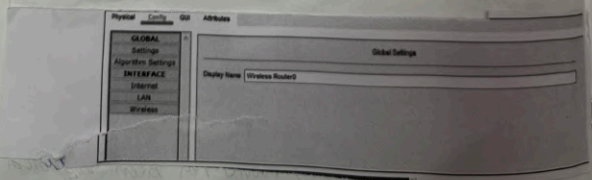


Part 1:

Build a sample network in the logical topology work space.

→ Step 1: Launch Packet tracer.

→ Step 2: Build the topology.

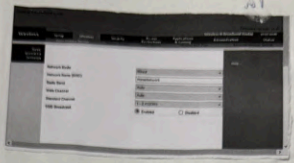


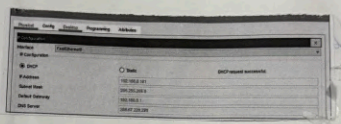
...networking
...nd internet

...al topology

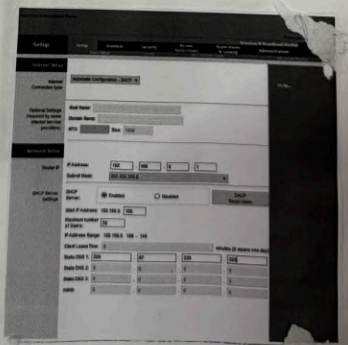
Part 2: Configure the network device

→ Step 1: Configure the wireless router

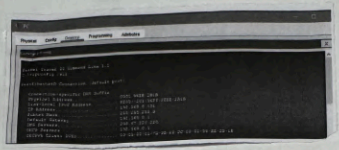




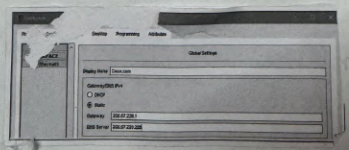
Step 3: Configure the laptop

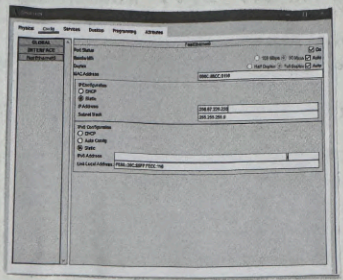


-> Step 4: Configure the PC

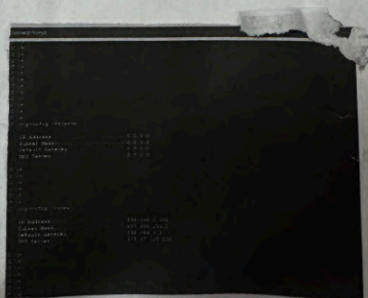


Step 5: Configure internet cloud.





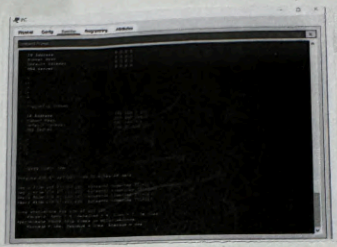
Step 6: Configure the CISCO.com server.



Part-3: Verify Connectivity

Step 1: Refresh the IPv4 setting on PC

a) Verify that PC is receiving IPv4 configuration information from DHCP



b) Test Connectivity to Cisco.com Server from PC

Result:

This experiment has been completed Successfully.

Experiment-11(b)

Aim: Simulate R+P using CISCO Packet trace.

Initial IP Configuration:

Device	Interface	IP Config	Connected with
PC0	Fast ethernet	10.0.0.2/8	Router 0's Fa0/1
Router 0	Fa 0/1	10.0.0.1/8	PC0's fast ethernet
Router 0	S0/0/1	192.168.1.254/30	Router 2's s0/0/1
Router 1	S0/0/1	192.168.1.254/30	Router 0's S2/0/0
Router 2	S0/0/1	192.168.1.256/30	Router 0's S0/0/1
PC	Fast ethernet	20.0.0.2/30	Router 2's Fa0/1

Assign IP address to PC

Double click PC's and click desktop menu item and click IP Config

Router 0:

Router (Config) # interface fast ethernet 0/0

Router (Config-if) # IP address 10.0.0.1

255.0.0.0 Router (Config) # no shutdown
Router (Config-if) # exit

Router 1:

Router > enable
Router # Configure
Router (Config) # interface serial 0/0/0
Router (Config-if) # no shutdown
Router (Config-if) # exit

Router 2:

Router > enable
Router # configure
terminal
Router (Config) # interface fast ethernet 0/0
Router (Config-if) # IP address 20.0.0.1
255.0.0.0 Router (Config-if) # no shutdown
Router (Config-if) # exit

Access the Command Prompt of PCs and use ping Command to test Connectivity from PC.

Result:

The experiment has been completed successfully.

Experiment: 12 (a)

Aim: Implement echo client server using TCP/UDP Sockets.

Algorithm:

TCP Server

1. Start the Program
2. Create server socket using a port numb
3. Wait using accept()
4. Create I/P & O/P stream
5. Receive message from client
6. Send same back to client
7. Stop

TCP - client

1. Start the Program
2. Create a Socket & Connect to server. using port no:
3. Create I/P O/P stream.
4. Read a message from user.

5. Send same to server
6. Receive the echoed message
7. Stop.

I/P

Enter message : Hello. server

O/P:

Server started
Waiting for client
Client Connected
Message from client : Hello Server.

Result:

The program has been executed successfully.

Experiment - 12(b)

Aim: To implement a chat client, server application using TCP/UDP sockets

Algorithm:

TCP chat server:

1. Start the Program.
2. Create server socket with
3. Wait using accept()
4. Create I/P & O/P stream
5. Receive message from client
6. Display message
7. Send reply to client
8. Close Socket & Stop Program.

TCP chat client:

1. Start the Program
2. Create Socket & Connect to server
3. Create I/P & O/P streams
4. Read a message from client server
5. Set message from client server.

6. Display
7. Close

I/P:

Client:
Server:
Client:
Server
Client

O/P:

Ser
Wai
Cl

Result

T
Suc

6. Display Reply.
7. Close the socket and stop program.

I/P:

Client : Hello
Server : Hi
Client : How are you?
Server : I am Fine
Client : Exit

O/P:

Server started
Waiting for Client
Client Connected

Result:

The Program has been executed succesfully.

Experiment-13

Aim: Implement your own ping program

Program:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int main() {
    int sock;
    char buffer[1024];
    struct sockaddr_in addr;
    struct icmphdr icmp;
    sock = socket(AF_INET, SOCK_RAW,
                  IPPROTO_ICMP);
    if(sock < 0) {
        perror("socket");
        return 1;
    }
    memset(&icmp, 0, sizeof(icmp));
    icmp.type = ICMP_ECHO;
    icmp.code = 0;
    sendto(sock, &icmp, sizeof(icmp), 0,
struct sockaddr*)&addr, sizeof(addr));
    printf("Ready
```

printf("

Result

The

printf("Reply received from 8.8.8.8\n");

Result:
The Program has been executed succesfully.

Experiment-14

Aim: Write a code using RAW socket to implement Packet sniffing.

Program:

```
#include<stdio.h>
#include<unistd.h>
#include<arpa/inet.h>
int main(){
    int sock;
    char buffer[65536];
    struct sockaddr saddr;
    int len = sizeof(saddr);
    while(1){
        int n; new from(sock, buffer,
size of buffer), &sadd.(Socklan_t*)&len
if(n>0){
    struct iphdr *ip = (struct iphdr*)buffer.
printf("Packet received\n");
}close(sock);
 return 0;
}
```

Result:

The program has been executed successfully.

Experiment - 15

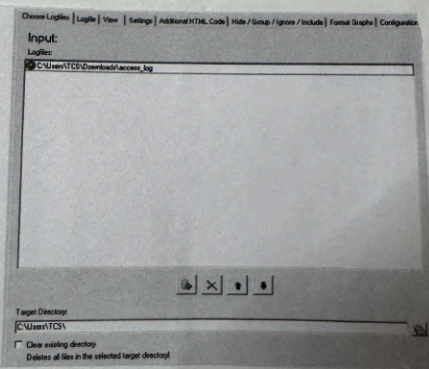
Aim: To analyze the different types of web logs using webalize tool.

Procedure:

Step 1: Run webalize window version

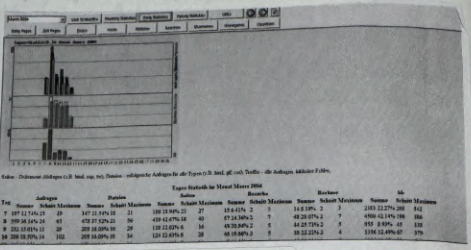
Step 2: Input web log file

Step 3: Press run webalizer



O/p:

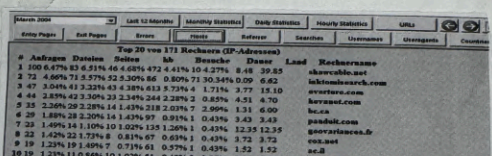
Monthly station:



Tages-Statistik im Monat März 2004

Tag	Anfragen Summe	Anfragen Schnitt	Anfragen Maximum	Dateien Summe	Dateien Schnitt	Dateien Maximum	Besuche Summe	Besuche Schnitt	Besuche Maximum	Rechner Summe	Rechner Schnitt	Rechner Maximum	kb Summe	kb Schnitt	kb Maximum
1	107	11.74%	28	89	167	11.93%	21	27	189	18.90%	21	27	15	6.01%	2
2	399	36.14%	24	63	472	27.12%	23	59	433	42.67%	32	40	87	24.36%	2
3	252	15.01%	17	29	208	16.09%	95	29	138	12.00%	4	16	69	25.84%	2
4	206	18.70%	14	102	208	16.09%	35	34	124	12.43%	8	28	66	18.98%	2
5	44	3.99%	3	3	44	3.40%	3	3	14	1.38%	2	3	3	1.03%	1
6	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
7	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
8	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
9	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
10	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
11	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
12	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
13	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
14	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
15	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
16	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
17	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
18	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
19	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
20	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
21	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
22	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
23	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
24	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
25	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
26	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
27	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
28	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
29	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
30	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0
31	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0	0	0	0.00%	0

Tabelle: Datenmenge Anfragen (z.B. html, asp, txt), Dateien - aufgerufen durch Anfragen für alle Typen (z.B. html, gif, css), Treffer - alle Anfragen, inklusive Fehler,



March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries

Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85		shawcable.net
2	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62		inktomisearch.com
3	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10		overture.com
4	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70		hevanet.com
5	35	2.26%	29	2.28%	14	1.43%	218	2.03%	7	2.99%	1.31	6.00		bc.ca
6	29	1.88%	28	2.20%	14	1.43%	97	0.91%	1	0.43%	3.43	3.43		pandait.com
7	23	1.49%	14	1.10%	10	1.02%	135	1.26%	1	0.43%	12.35	12.35		geovariances.fr
8	22	1.42%	22	1.73%	8	0.81%	67	0.63%	1	0.43%	3.72	3.72		cnc.net
9	19	1.23%	19	1.49%	7	0.71%	61	0.57%	1	0.43%	1.52	1.52		ac.il
10	19	1.23%	11	0.86%	10	1.02%	51	0.48%	2	0.85%	3.04	5.87		netinfo.bg
11	15	0.97%	14	1.10%	13	1.32%	130	1.22%	3	1.28%	1.29	3.87		e-taxi
12	13	0.84%	13	1.02%	13	1.32%	120	1.12%	1	0.43%	3.88	3.88		telia.net
13	13	0.84%	13	1.02%	1	0.10%	28	0.26%	1	0.43%	0.13	0.13		net.ar
14	13	0.84%	13	1.02%	2	0.20%	30	0.28%	1	0.43%	0.87	0.87		dkl.com
15	12	0.78%	11	0.86%	9	0.92%	68	0.63%	1	0.43%	25.53	25.53		fiscali.de
16	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.33	0.33		whts.org
17	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.10	0.10		qwest.net
18	11	0.71%	11	0.86%	7	0.71%	41	0.38%	2	0.85%	0.56	1.03		rndtest.net
19	10	0.65%	9	0.71%	7	0.71%	41	0.38%	1	0.43%	1.40	1.40		net.au
20	10	0.65%	10	0.78%	2	0.20%	61	0.57%	1	0.43%	0.17	0.17		3_243_it_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10		overture.com
2	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85		shawcable.net
3	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70		hevanet.com

Result:

The Experiment has been executed succesfully